

JOB RECOMMENDATION SYSTEM USING ML ALGORITHM

A project report submitted in partial fulfillment of the requirements

for the award of credits to

Machine Learning Course of

Bachelor of Technology

In

DEPARTMENT OF ELECTRONICS AND COMMUNICATION

ENGINEERING - (ECE)

By

P. KEERTHI SAI

23BQ1A04C3

N. PRASANNA

23BQ1A04B0

L. JYOTHI

23BQ1A0486



DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

VASIREDDY VENKATADRI INSTITUTE OF TECHNOLOGY

(Approved by AICTE and permanently affiliated to JNTUK)

Accredited by NBA and NAAC with 'A' Grade

NAMBUR (V), PEDAKAKANI (M), GUNTUR-522 508

DEPARTMENT OF ELECTRONICS AND COMMUNICATION
ENGINEERING - (ECE)
VASIREDDY VENKATADRI INSTITUTE OF TECHNOLOGY: NAMBUR
JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY KAKINADA



CERTIFICATE

This is to certify that the project titled “**JOB RECOMMENDATION SYSTEM USING ML**” is a bonafide record of work done by **P. KEERTHI SAI(23BQ1A04C3)**, **L. JYOTHI(23BQ1A0486)**, **N. PRASANNA(23BQ1A04B0)** under the guidance of **Mrs. Ramya Harika, Asst Professor** in partial fulfillment of the requirement for the award of credits to **Machine Learning** course of Bachelor of Technology in **ELECTRONICS AND COMMUNICATION ENGINEERING, JNTUK** during the academic year 2025-26.

Mrs. Ramya Harika
Course Instructor

Dr.Sk.Enaul Haq Shaik
Head of the Department

DECLARATION

We are **P.Keerthi Sai(23bq1a04c3),L. Jyothi(23bq1a0486),N. Prasanna(23bq1a04b0)** hereby declare that the project report entitled “**Job Recommendation using ML Algorithm**” done by us under the guidance of **Mrs.Ramya Harika, Assistant Professor, Department of Electronics and Communication Engineering** is submitted by partial fulfilment of requirements for the award of degree of **BACHELOR OF TECHNOLOGY** in **ELECTRONICS AND COMMUNICATION ENGINEERING**

DATE:

PLACE: VVITU, NAMBUR

Signature of the candidates

P.Keerthi Sai (23bq1a04c3)

L. Jyothi(23bq1a0486)

N. Prasanna(23bq1a04b0)

ACKNOWLEDGEMENT

We express our sincere thanks wherever it is due

We express our sincere thanks to the Chairman, **Vasireddy Venkatadri Institute of Technology, Sri Vasireddy Vidya Sagar** for providing us well equipped infrastructure and environment.

We thank **Dr. Y. Mallikarjuna Reddy**, Principal, Vasireddy Venkatadri Institute of Technology, Nambur, for providing us the resources for carrying out the project.

We express our sincere thanks to **Dr. K. Giribabu**, Dean of Academics for providing support and stimulating environment for developing the project.

Our sincere thanks to **Dr.Sk.Enaul Haq Shaik**, Head of the Department, Department of ECE, for his co-operation and guidance which helped us to make our project successful and complete in all aspects.

We also express our sincere thanks and are grateful to our guide **Ms. Ramya Harika**, Asst. Professor, Department of ECE, for motivating us to make our project successful and fully complete. We are grateful for his precious guidance and suggestions.

We also place our floral gratitude to all other teaching staff and lab technicians for their constant support and advice throughout the project.

P.KEERTHI SAI

(23BQ1A04C3)

L.JYOTHI

(23BQ1A0486)

N. PRASANNA

(23BQ1A04B0)

TABLE OF CONTENTS

Chapter No.	Title	Page No.
-	Abstract	vi
Chapter 1	Introduction	
1.1	Problem Statement	1
1.2	Objectives of the Project	2
1.3	Project Aims	3
1.4	Project Scope	4
Chapter 2	Software Requirements	
2.1	Programming Language	5
2.2	Libraries and Frameworks	6
2.3	Development Environment	7
2.4	Operating System	8
Chapter 3	Literature Review	
3.1	Importance of Machine Learning in Household electricity	12
3.2	Review of Related Work	13
3.3	Gaps in Literature	14
3.4	Relevance to the Present Work	15
Chapter 4	Implementation	
4.1	Project Structure	19
4.2	Algorithm Used in the Project	20
Chapter 5	Results and Output	
5.1	Output Screens and Analysis	22
5.2	Evaluation Metrics and Accuracy	22
Chapter 6	Conclusion	
6.1	Summary	29
6.2	Future Enhancements	29
Chapter 7	References	30

ABSTRACT

In today's competitive job market, connecting the right candidates with the right job opportunities efficiently is crucial. Traditional job portals often fail to provide personalized recommendations, leading to inefficient job search experiences. This paper presents a Job Recommendation System leveraging Machine Learning (ML) algorithms to enhance the relevance and accuracy of job suggestions. The system analyzes user resumes, preferences, and selected locations to predict and recommend suitable job roles. Key stages include data collection, exploratory data analysis (EDA), preprocessing, feature extraction using techniques such as TF-IDF for text and encoding for categorical data, and implementation of classification models including Logistic Regression, Random Forest, and Gradient Boosting. Model performance is evaluated using accuracy, precision, recall, and F1-score metrics. The system demonstrates high precision in matching candidates to job profiles, improving both user satisfaction and recruiter efficiency. This ML-driven approach not only personalizes recommendations but also adapts over time, offering a scalable solution for modern recruitment platforms.

Chapter 1

INTRODUCTION

1.1. Problem statement :

In the fast-evolving digital job market, job seekers are presented with an overwhelming number of job postings across various online platforms. Navigating this vast pool to identify roles that truly match their skills, experience, and career aspirations is often a frustrating and time-consuming process. At the same time, recruiters struggle to identify the most suitable candidates from a large volume of applications, many of which are irrelevant or underqualified.

Traditional job recommendation systems rely primarily on keyword-based searches, rule-based filters, or basic matching algorithms that do not consider the contextual or semantic relationships between user profiles and job requirements. These systems lack personalization and adaptability, often leading to inaccurate, redundant, or generic job suggestions. This inefficiency results in poor user experience, low application conversion rates, and reduced effectiveness of the recruitment process.

The core problem is the **lack of an intelligent, data-driven approach** that can understand and interpret user preferences, skills, and job requirements in a meaningful way. Therefore, there is a need to build a **Job Recommendation System using Machine Learning algorithms** that can:

- Analyze structured data (e.g., job titles, locations, education, experience) and unstructured data (e.g., resumes, job descriptions) using Natural Language Processing (NLP) techniques.
- Predict the most relevant job roles for each user based on their profile and historical behavior.
- Continuously learn from user interactions (clicks, applications, feedback) to improve future recommendations.
- Handle cold-start scenarios where new users or new jobs are added to the system.

The objective is to design a recommendation engine that not only improves the relevance and accuracy of job suggestions but also enhances the overall job-seeking and hiring experience for both candidates and employers.

1.2. Objectives of the project:

The primary objective of this project is to design and implement an intelligent Job Recommendation System using Machine Learning (ML) algorithms that can accurately and efficiently match job seekers with suitable job opportunities. The system aims to enhance the job search experience by providing personalized, relevant, and data-driven job recommendations. Below are the detailed objectives:

1. Data Collection and Integration

- To collect a comprehensive dataset that includes job listings, job descriptions, required qualifications, and user profiles (resumes, skills, education, experience, etc.).
- To integrate data from multiple sources such as job portals, resume databases, or simulated datasets for training and testing purposes.

2. Data Preprocessing and Cleaning

- To clean and preprocess the data by removing inconsistencies, handling missing values, and converting unstructured text into structured format.
- To apply text normalization techniques such as tokenization, stemming, lemmatization, and stop-word removal to prepare textual data for analysis.

3. Feature Extraction and Engineering

- To extract meaningful features from job descriptions and resumes using Natural Language Processing (NLP) techniques such as TF-IDF, Bag-of-Words, or Word Embeddings (e.g., Word2Vec, BERT).
- To encode categorical variables (e.g., job type, location, degree) using methods like Label Encoding or One-Hot Encoding for use in ML models.

4. Model Selection and Implementation

- To experiment with various supervised learning algorithms such as Logistic Regression, Random Forest, Support Vector Machines (SVM), and Gradient Boosting for job-role classification or prediction.
- To optionally explore unsupervised or hybrid approaches (e.g., K-Means Clustering, Collaborative Filtering) for recommendation in the absence of labeled data.

5. Model Evaluation and Optimization

- To evaluate the performance of each model using metrics like accuracy, precision, recall, F1-score, and confusion matrix.
- To perform hyperparameter tuning and cross-validation to improve model generalization and reduce overfitting.

6. Job Matching and Recommendation Generation

- To implement a recommendation engine that computes similarity scores or prediction probabilities to rank and recommend the most relevant job roles to each user.
- To personalize recommendations based on user preferences, history, and profile-job compatibility.

7. Cold Start Handling and Adaptability

- To design fallback strategies for new users or new job postings (cold start problem) using content-based filtering or profile similarity approaches.
- To enable the system to learn and adapt over time using feedback loops (e.g., implicit feedback like click behavior or explicit ratings).

8. Visualization and Reporting

- To generate insightful visualizations for user profiles, job trends, model performance, and recommendation accuracy using tools like Matplotlib, Seaborn, or Plotly.
- To present reports or dashboards summarizing system behavior, prediction results, and user engagement.

1.3. Project Aims:

The primary aim of this project is to design and develop an intelligent **Job Recommendation System** that leverages **Machine Learning algorithms** to provide accurate, relevant, and personalized job suggestions to users based on their profile, skills, experience, and preferences. This system is intended to improve the efficiency and effectiveness of the job search and recruitment process for both job seekers and employers.

Specific Aims

1. **To automate the job recommendation process** by replacing traditional keyword-based filtering methods with intelligent, data-driven ML models.
2. **To improve the quality and relevance of job recommendations** by analyzing and learning from structured and unstructured data such as resumes and job descriptions.
3. **To reduce the time and effort** job seekers spend searching for appropriate job openings by providing personalized recommendations based on profile-job matching.
4. **To utilize Natural Language Processing (NLP)** techniques to extract meaningful information from textual data in resumes and job postings.
5. **To explore and evaluate various machine learning algorithms** such as Logistic Regression, Random Forest, Gradient Boosting, and Support Vector Machines for classification and prediction.
6. **To design a model capable of ranking job postings** based on the similarity and compatibility with a user's profile and preferences.
7. **To develop a system that can adapt over time**, incorporating user interactions and feedback to refine and improve future recommendations.
8. **To handle cold-start scenarios** by using content-based methods to provide reasonable recommendations for new users or newly added job postings.
9. **To enhance the decision-making process for recruiters**, by filtering out the most relevant candidates based on automated resume-job matching.
10. **To lay the groundwork for a scalable, real-world deployment** of a job recommendation platform that could integrate with online job portals or career platforms.

1.4. Project Scope:

The scope of this project encompasses the design, development, and evaluation of a **Job Recommendation System** that utilizes **Machine Learning (ML)** techniques to provide personalized and relevant job suggestions to users based on their profiles, skills, and preferences. The system aims to assist job seekers in finding suitable employment

opportunities efficiently and accurately, while also helping recruiters identify appropriate candidates

In-Scope Activities

1. Data Collection and Preparation

- Collecting a dataset consisting of job descriptions, user profiles, resumes, skills, and job categories.
- Preprocessing and cleaning of raw data to ensure quality input for ML models.

2. Feature Engineering and Text Processing

- Extracting features from resumes and job descriptions using NLP techniques such as TF-IDF or word embeddings.
- Encoding categorical data like job titles, locations, and skill levels.

3. Model Development

- Implementation of ML algorithms such as Logistic Regression, Random Forest, Gradient Boosting, or other classifiers to predict job-role relevance.
- Comparison of model performance using standard evaluation metrics (accuracy, precision, recall, F1-score).

4. Recommendation Generation

- Development of a recommendation engine to suggest job roles ranked by relevance based on the user's profile and input.
- Implementation of similarity-based or prediction-based ranking mechanisms.

5. User Profile Matching

- Matching users to job postings based on features extracted from resumes and preferences like location, domain, or experience level.

6. Performance Evaluation

- Testing the system using historical or simulated user data.
- Evaluating recommendations for correctness, diversity, and relevance.

7. Visualization and Reporting

- Visual representation of results, user-job matching trends, and model performance using tools like Matplotlib or Seaborn.

Chapter 2

SOFTWARE REQUIREMENTS

The development of the Job Recommendation System using Machine Learning requires a robust and flexible software environment to support data processing, machine learning model implementation, and visualization. The project will be developed using **Python 3.7 or above**, chosen for its simplicity and rich ecosystem of data science libraries. The preferred operating system is **Windows 10**, although Linux distributions like **Ubuntu** or **macOS** can also be used effectively.

For coding and experimentation, **Jupyter Notebook** and **Google Colab** will be used due to their interactive nature and ease of visualizing data and model outputs. Alternatively, **Visual Studio Code** may be used for writing modular scripts and debugging. To manage Python packages and environments, the **Anaconda Distribution** will be helpful.

Several Python libraries are essential for the project. For data handling and preprocessing, libraries like **pandas** and **numpy** are used. To implement machine learning algorithms such as Logistic Regression, Random Forest, or Gradient Boosting, **scikit-learn** is the primary library. If gradient boosting is used for performance optimization, additional libraries like **XGBoost** or **LightGBM** may be included. For handling and extracting features from textual data in resumes and job descriptions, **Natural Language Processing (NLP)** libraries like **NLTK**, **SpaCy**, and **TF-IDF Vectorizer** from scikit-learn are used. **Gensim** may also be employed for word embeddings if needed.

2.1 Programming Language

- The primary programming language used for the development of the Job Recommendation System is **Python**. Python is widely regarded as the most suitable language for data science and machine learning applications due to its simplicity, readability, and vast collection of libraries and frameworks. It offers powerful tools for data preprocessing, model building, evaluation, and deployment. Python's support for libraries such as **scikit-learn** for machine learning, **pandas** and **numpy** for data manipulation, and **NLTK** or **SpaCy** for natural

language processing makes it an ideal choice for this project. Additionally, Python integrates easily with visualization libraries like **matplotlib** and **seaborn**, and with web frameworks like **Flask** or **Streamlit**, allowing for end-to-end system development from data processing to user interface.

2.2 Libraries and Frameworks

1. Data Handling and Preprocessing

- **Pandas:**

Pandas is a powerful data manipulation and analysis library. It provides data structures like DataFrames, which make it easy to load, clean, filter, and transform job and resume data. For instance, you can use it to merge datasets, handle missing values, and analyze trends.

- **NumPy:**

NumPy is used for numerical computations and is especially useful when dealing with arrays or performing mathematical operations on large datasets. It complements Pandas in efficient handling of numeric data.

2. Natural Language Processing (NLP)

- **NLTK (Natural Language Toolkit):**

This library is commonly used for processing textual data like resumes and job descriptions. It provides tools for tokenization, stemming, stopword removal, and part-of-speech tagging.

- **spaCy:**

spaCy is a fast and efficient NLP library ideal for industrial use. It offers robust support for tasks like named entity recognition, dependency parsing, and word vector representation. It's often preferred for its speed and ease of use.

- **scikit-learn's TfidfVectorizer**

These are used to convert textual data into numerical features by representing text documents as vectors based on word frequency or importance (TF-IDF). These features are then used by ML algorithms for training and prediction.

- **Gensim:**

Gensim is used for topic modeling and word embeddings like Word2Vec. This

helps in capturing semantic meaning from text data and improves the accuracy of recommendations based on user profiles and job descriptions.

3. Machine Learning and Model Building

- **scikit-learn:**

A widely-used library that provides tools for classification, regression, clustering, model selection, and evaluation. It supports algorithms like Logistic Regression, Decision Trees, Random Forests, and more. It also has utilities for data preprocessing and splitting data into training and testing sets.

- **XGBoost / LightGBM:**

These are high-performance gradient boosting frameworks that are particularly effective for structured/tabular data. They offer fast training speed and high accuracy, making them suitable for large-scale recommendation systems.

- **TensorFlow / Keras:**

These libraries are used for building deep learning models such as neural networks and autoencoders. Keras is a high-level API that runs on top of TensorFlow and simplifies building and training complex models.

- **PyTorch:**

An alternative to TensorFlow, PyTorch is known for its flexibility and dynamic computation graph. It's useful when custom model architectures are needed, such as hybrid content-based and collaborative filtering models.

4. Data Visualization

- **Matplotlib:**

A basic plotting library for creating static, animated, and interactive plots. It's often used for visualizing distributions, model performance, and EDA (Exploratory Data Analysis).

- **Seaborn:**

Built on top of Matplotlib, Seaborn simplifies the creation of attractive statistical plots. It is especially useful for heatmaps, box plots, and pair plots during EDA.

- **Plotly:**

An advanced library for creating interactive graphs and dashboards. It can be used to develop user-friendly interfaces or dashboards for recruiters or job seekers.

5. Model Saving and Utility Tools

- **Joblib / Pickle:**

These libraries are used for serializing and saving trained models so that they can be reused later without retraining. Joblib is preferred for large NumPy arrays and models.

- **Yellowbrick:**

A visualization tool that works with scikit-learn to evaluate and visualize machine learning models. It helps in better understanding model performance and making informed decisions.

6. Deployment and Web Integration (Optional)

- **Streamlit:**

A Python framework used to quickly build and deploy interactive web apps. It's ideal for creating a front-end interface for your job recommendation system without needing extensive web development skills.

- **Flask / Django:**

These are Python web frameworks used for building and deploying full-scale web applications. Flask is lightweight and easy to use, while Django is more feature-rich and suited for larger applications.

- **SQLite / PostgreSQL / MongoDB:**

These are databases used to store user data, job listings, and application history. SQL-based databases are useful for structured data, while MongoDB is better suited for semi-structured or unstructured data.

- **BeautifulSoup / Selenium / Scrapy:**

These are web scraping libraries used to collect job listings from websites if real-time or up-to-date data is required for the recommendation engine.

2.3 Development Environment

- **1. Operating System**
- **Windows / macOS / Linux (Ubuntu preferred for ML tasks)**
Most libraries work cross-platform, but Linux (especially Ubuntu) is often preferred for compatibility, performance, and deployment purposes.
-
- **2. Programming Language**
- **Python (version 3.7 or higher)**
Python is the primary language for machine learning and data science due to its rich ecosystem of libraries and community support.
- **3. IDE / Code Editors**
- **Jupyter Notebook**
Ideal for exploratory data analysis (EDA), model development, and visualization. It allows you to write and execute code in blocks, making debugging and testing easier.
- **VS Code (Visual Studio Code)**
A lightweight and powerful code editor with Python support, Git integration, and extensions for Jupyter Notebooks and ML development.
- **PyCharm**
A full-featured IDE tailored for Python. Best suited for larger projects and teams needing advanced features like debugging, version control, and virtual environments.
- **4. Python Package Manager**
- **pip**
Default Python package installer. Use it to install required libraries like scikit-learn, pandas, etc.
- **conda (Anaconda / Miniconda)**
An environment manager and package installer. Helps create isolated environments with specific Python and library versions to avoid conflicts.
- **5. Environment Management**
- **Anaconda Navigator / conda environments**
Allows creating clean environments for different projects. Useful for managing dependencies and avoiding version issues.

- **virtualenv**
Lightweight alternative to conda for creating isolated Python environments.
- **6. Libraries and Dependencies**
- You'll need to install the following Python packages:
- bash
- CopyEdit
- pip install pandas numpy scikit-learn matplotlib seaborn nltk spacy gensim
joblib flask streamlit
- (Optional: TensorFlow, PyTorch, XGBoost, LightGBM, Plotly, etc.)
- **7. Version Control System**
- **Git**
Used to track code changes and collaborate with others. You can host your code on:
- **GitHub**
- **GitLab**
- **Bitbucket**
- **8. Data Storage / Database (Optional)**
- **CSV / Excel files**
Simple flat-file formats for storing job and resume data during development.
- **SQLite / PostgreSQL / MySQL**
If your system scales to handle user data or job listings, relational databases are useful.
- **MongoDB**
A NoSQL database, suitable if you're working with semi-structured data like JSON resumes.

2.4 Operating System

The project is designed to be platform-independent and can be developed and executed on multiple operating systems, ensuring broad accessibility and flexibility. The key operating systems compatible with this project include:

- **Windows:**
Widely used in personal computers, Windows supports all the necessary tools

and libraries required for Python development. Users can install Python distributions such as the official Python installer or Anaconda, which simplifies package management and environment setup. Popular IDEs like VS Code, PyCharm, and Jupyter Notebook run smoothly on Windows, making it a common choice for development.

- **Linux:**

Linux distributions (such as Ubuntu, Fedora, and CentOS) are highly favored for data science and machine learning due to their robustness, security, and native support for many programming tools. Linux provides efficient command-line utilities for managing Python environments and dependencies. It is also widely used in server environments where machine learning models are deployed.

Chapter 3

LITERATURE REVIEW

3.1 Importance of Machine Learning in Job recommendation

- A **Job Recommendation System** is a valuable tool in today's digital hiring ecosystem, designed to bridge the gap between job seekers and employers by intelligently matching candidates to relevant job opportunities. Its importance lies in its ability to streamline, personalize, and optimize the job search and recruitment process.
- **1. Efficient Job Matching**
 - Job recommendation systems use machine learning and data analysis to suggest roles that closely match a user's profile, skills, and interests. This reduces the time and effort required to sift through thousands of job listings manually.
- **2. Personalized User Experience**
 - By analyzing user behavior, past applications, search history, and profile details, the system tailors recommendations to individual preferences. This personalization increases the chances of user satisfaction and job application success.
- **3. Enhanced Employer Reach**

- Employers benefit as their job postings are shown to the most relevant candidates. This targeted visibility leads to higher-quality applications and reduces time-to-hire.
- **4. Increased User Engagement**
- When users receive relevant job suggestions, they are more likely to stay engaged with the platform. A smart recommendation engine keeps users returning to check for new opportunities, improving retention.
- **5. Scalability and Automation**
- In large-scale job portals or recruitment platforms, manual filtering is not feasible. A recommendation system automates and scales the matching process, ensuring consistent and timely suggestions for millions of users.
- **6. Reduction in Information Overload**
- With thousands of listings posted daily, job seekers can feel overwhelmed. Recommendation systems filter out irrelevant jobs and highlight only the most pertinent ones, improving decision-making.
- **7. Data-Driven Insights**
- These systems collect valuable data about market trends, in-demand skills, and user preferences, which can be used by job platforms, employers, and candidates to make informed decisions.
- **8. Career Growth and Skill Alignment**
- Advanced systems may suggest roles based on emerging trends or similar career paths, encouraging users to upskill or consider alternative opportunities they hadn't thought of—thus aiding long-term career development.
- **Real-World Examples:**
- **LinkedIn:** Recommends jobs based on your network, profile, and browsing history.
- **Indeed / Naukri / Glassdoor:** Uses user interactions and resume data to push personalized job listings.
- **Google for Jobs:** Aggregates listings across platforms and ranks them using AI.
- **In Summary:**
- A job recommendation system is **essential in modern recruitment** because it improves **accuracy, speed, personalization, and user satisfaction**—benefiting

both **job seekers and recruiters** in an increasingly competitive job market.

3.2 Review of Related Work

Over the years, numerous research studies and practical implementations have been conducted to enhance job recommendation systems using machine learning techniques. Traditional systems relied heavily on keyword-based or rule-based matching, which often failed to capture the contextual relevance of job descriptions and candidate profiles. To overcome these limitations, content-based filtering and collaborative filtering techniques were introduced. Content-based filtering uses the attributes of job postings and resumes, such as skills, experience, and education, to make recommendations, whereas collaborative filtering suggests jobs based on the preferences of similar users. Hybrid approaches combining both methods have shown improved accuracy and personalization. For example, LinkedIn and Glassdoor use collaborative filtering along with behavioral data to recommend relevant job listings.

Recent advancements in natural language processing (NLP) and deep learning have further enhanced job recommendation systems. Researchers have employed techniques such as Word2Vec, TF-IDF, and BERT to extract semantic features from textual data like resumes and job descriptions. These embeddings help in better understanding and matching of user profiles to job listings beyond exact keyword matches. Some studies also explore the use of autoencoders and deep neural networks for modeling complex relationships in user-job interactions. Moreover, recommender systems have incorporated user feedback loops and learning-to-rank models to dynamically improve results based on click behavior and application history. Despite these advancements, challenges like cold-start problems (for new users or jobs), data sparsity, and real-time adaptability remain key areas of ongoing research. Overall, existing work lays a strong foundation and continues to evolve toward more intelligent, scalable, and personalized job recommendation solutions.

3.3 Gaps in Literature

Despite significant advancements in job recommendation systems using machine learning, several gaps still exist in the current body of literature. One of the most prominent challenges is the **cold-start problem**, where the system struggles to make accurate recommendations for new users or newly posted jobs due to a lack of prior data. Additionally, many existing systems rely heavily on structured data such as job titles, skills, and locations, while often **neglecting unstructured data** like resumes and cover letters, which contain valuable context and nuanced information. Another gap lies in the **limited integration of real-time feedback mechanisms**; few systems adapt dynamically based on user behavior such as clicks, applications, or rejection patterns, which can hinder personalization over time. Furthermore, while deep learning and NLP techniques have improved semantic understanding, they are often **computationally expensive and difficult to interpret**, making them less accessible for smaller platforms or applications requiring transparency. Many studies also lack diversity in datasets, often focusing on limited regions or industries, which reduces the generalizability of their models. Lastly, **ethical concerns and bias** in recommendation outcomes are rarely addressed in depth, even though biased suggestions can significantly impact fair hiring practices. These gaps present important opportunities for future research and development in building more accurate, inclusive, and adaptive job recommendation systems.

3.4 Relevance to the Present Work

The present work draws significant relevance from existing research and developments in the field of job recommendation systems but aims to enhance and tailor these ideas to meet current technological and user-centric demands. While previous systems have successfully implemented basic filtering techniques and collaborative models, they often fall short in addressing personalization, real-time adaptability, and the effective use of both structured and unstructured data. This project builds upon those foundations by integrating advanced machine learning techniques with natural language processing to better analyze resumes and job descriptions. In doing so, it moves beyond simple keyword matching and incorporates semantic understanding, which improves the accuracy of recommendations. Furthermore, this system is designed to include user interaction feedback to continuously refine and personalize job suggestions, which many existing systems overlook. By implementing and comparing multiple ML algorithms, the project contributes to

identifying effective and scalable solutions for job matching. Thus, the present work not only aligns with ongoing efforts in the domain but also addresses the limitations of past research, offering a more intelligent, flexible, and user-adaptive job recommendation system.

Chapter 4

IMPLEMENTATION

The project is systematically structured to follow a complete machine learning pipeline, ensuring that each stage contributes effectively to the development of a reliable electricity consumption forecasting model. The workflow is divided into several key components, as outlined below:

1. Data Collection

Data collection is the foundational step in developing a job recommendation system, as the quality and structure of the data directly influence the system's performance. In this project, data is collected from publicly available sources such as job portals (e.g., Kaggle, Indeed, Naukri.com datasets) or professional networking platforms like LinkedIn. The dataset typically includes detailed information on job listings such as job titles, descriptions, required skills, locations, companies, and salary ranges. In addition to job data, user-related data such as resumes, skills, educational background, past job experience, and preferences are also gathered when available. If user profiles are not available, synthetic profiles can be created to simulate real-world behavior. Web scraping techniques using Python libraries like **BeautifulSoup** or **Scrapy** may be employed to extract data directly from online sources, ensuring that a diverse range of job roles and industries is represented. The collected data is stored in structured formats such as CSV or JSON files, which are later used for analysis, preprocessing, and model training. Ensuring that the dataset is rich, diverse, and representative of real-world scenarios is critical for building an effective and scalable job recommendation

2. Data Preprocessing

Data preprocessing is a crucial step that ensures the dataset is clean, consistent, and ready for machine learning model training. After collecting the raw data, it often contains noise such as missing values, duplicate records, irrelevant information, and inconsistent

formatting. The first step in preprocessing involves handling missing data by either removing incomplete rows or imputing values using statistical methods such as mean, median, or mode. Duplicate entries are eliminated to avoid bias in the model. Text fields like job titles, descriptions, and skills are standardized by converting all characters to lowercase and removing special characters, punctuation, and stop words. Tokenization, stemming, or lemmatization techniques are applied to further normalize the text. For converting text into numerical form, Natural Language Processing (NLP) techniques such as **TF-IDF**, **Bag of Words**, or **Word2Vec** are used to transform job descriptions and resumes into vector representations. Categorical data like job types or locations is encoded using **label encoding** or **one-hot encoding**. The final preprocessed dataset is then scaled and split into training and testing sets to ensure the machine learning models receive accurate and well-structured input. This preprocessing phase is essential to improve the performance, accuracy, and generalizability of the job recommendation system.

3. Feature Engineering

Feature engineering is a vital process in building an effective job recommendation system, as it involves transforming raw data into meaningful features that can improve the predictive power of machine learning models. In this step, relevant attributes are selected and constructed from the preprocessed data to represent both job listings and user profiles. For job postings, features such as job title, required skills, location, experience level, and job description are extracted and encoded. Similarly, for user profiles, features like education level, technical and soft skills, past experience, and preferred job location are considered. Textual fields like skills and job descriptions are converted into numerical vectors using **TF-IDF (Term Frequency-Inverse Document Frequency)** or **Word2Vec**, which capture the semantic relevance between user skills and job requirements. Additionally, categorical variables like job sector or employment type are transformed using **one-hot encoding** or **label encoding**. Interaction-based features such as user click behavior, application history, or time spent on job listings can also be engineered if available, to help personalize recommendations. The goal of feature engineering is to ensure that both job and user data are represented in a way that captures their key characteristics and relationships, enabling the machine learning model to make accurate and meaningful job recommendations.

4. Modeling

In the modeling phase of the job recommendation system, various machine learning algorithms are applied to learn patterns from the prepared dataset and generate meaningful job suggestions for users. The process begins with selecting suitable models based on the nature of the problem—classification or recommendation. For classification tasks, algorithms such as **Logistic Regression**, **Random Forest**, **Support Vector Machine (SVM)**, and **K-Nearest Neighbors (KNN)** are trained to predict the most appropriate job category for a user based on their resume features. In recommendation settings, techniques like **content-based filtering** are employed, where job descriptions and resumes are represented as vectors using **TF-IDF** or **Word2Vec**, and **cosine similarity** is calculated to rank jobs based on their closeness to the user profile. Additionally, **collaborative filtering** can be used to suggest jobs based on preferences and behaviors of similar users. For each model, hyperparameter tuning is performed using techniques like **Grid Search** and **Cross-Validation** to enhance accuracy and prevent overfitting. The output from each model is evaluated using appropriate metrics, and the best-performing model is selected for integration into the final system. This modeling step is central to the system's ability to provide relevant and personalized job recommendations to users.

5. Evaluation

The evaluation phase is essential to assess the effectiveness and reliability of the job recommendation system. After training the machine learning models, their performance is measured using standard evaluation metrics to determine how accurately and efficiently they can suggest relevant jobs to users. Key metrics include **accuracy**, which indicates the overall correctness of predictions; **precision**, which measures how many of the recommended jobs are truly relevant; **recall**, which evaluates how well the model retrieves all the suitable jobs for a user; and the **F1-score**, which provides a balance between precision and recall. A confusion matrix is often used to visualize the performance of classification models, showing true positives, false positives, true negatives, and false negatives. For recommendation-based models using similarity scores, additional metrics like **Mean Average Precision (MAP)**, **Mean Reciprocal Rank (MRR)**, or **Normalized Discounted Cumulative Gain (nDCG)** may be used to assess the quality of ranked job recommendations. **Cross-validation** is applied to ensure the model performs well across different subsets of the data and is not overfitted to a particular set. By comparing the results of different models, the most effective algorithm is selected for final deployment. This comprehensive evaluation ensures that the system can deliver accurate, relevant, and

personalized job recommendations to users in real-world scenarios.

6. Visualization

Visualization plays a crucial role in understanding the dataset, analyzing model performance, and interpreting the results of the job recommendation system. During the exploratory data analysis (EDA) stage, various graphs are created to gain insights into the dataset's structure and composition. For example, **bar charts** and **pie charts** are used to show the distribution of jobs across different industries, locations, and required skills. **Word clouds** help visualize the most frequent terms in job descriptions and user resumes, giving an intuitive understanding of dominant skills and roles. **Correlation heatmaps** are generated to identify relationships between different numerical features, such as experience level and job category.

After training the models, **confusion matrices** and **ROC curves** are used to assess classification performance, allowing us to visually compare model accuracy, precision, and recall. In recommendation models, **scatter plots** and **similarity heatmaps** illustrate how close job vectors are to user vectors based on cosine similarity scores. Additionally, **performance comparison charts** like line graphs or grouped bar plots are used to compare different models based on accuracy, F1-score, and other evaluation metrics. These visualizations not only aid in debugging and refining the models but also help present the outcomes clearly to non-technical stakeholders, enhancing the interpretability and impact of the system.

4.1. Project Structure:

The project structure of the Job Recommendation System is organized in a modular and scalable manner to ensure clarity, maintainability, and ease of development. At the top level, the project directory contains separate folders for **data**, **notebooks**, **scripts**, **models**, and **outputs**. The **data** folder holds raw and preprocessed datasets, including CSV or JSON files of job listings and user resumes. The **notebooks** directory includes Jupyter notebooks used for exploratory data analysis (EDA), visualization, and experimentation with various models. The **scripts** folder contains reusable Python modules for tasks like data preprocessing, feature extraction, model training, evaluation, and recommendation logic. Trained models and serialized objects (e.g., using `joblib` or `pickle`) are saved in

the **models** directory for easy loading during deployment. The **outputs** folder stores generated reports, graphs, evaluation results, and logs.

In addition, the project includes a **main.py** or **app.py** file that serves as the entry point for the application. If the system has a web interface, a **templates** folder (for HTML) and **static** folder (for CSS/JS files) are included for frontend components—especially when using frameworks like Flask or Streamlit. A **requirements.txt** or **environment.yml** file lists all dependencies, while a **README.md** provides documentation on setup and usage. This structured approach supports efficient collaboration, testing, and future enhancements, making the system robust and ready for deployment.

4.2. Algorithm Used in the Project :

1. Content-Based Filtering using TF-IDF + Cosine Similarity

This algorithm recommends jobs to a user by comparing the user's resume or skillset with job descriptions and ranking jobs based on similarity.

Step-by-Step:

Step 1: Input Data

- User resumes (text form)
- Job descriptions (text form)
- Additional features: skills, experience, location (optional)

Step 2: Text Preprocessing

- Convert all text to lowercase
- Remove stopwords (like “the”, “and”, “a”, etc.)
- Remove punctuation and special characters
- Apply stemming or lemmatization (e.g., “running” → “run”)

Step 3: Vectorization using TF-IDF

Use TF-IDF (Term Frequency–Inverse Document Frequency) to convert text data into numerical vectors:

- **TF**: How often a term appears in a document
- **IDF**: How unique that term is across all documents
- Output: a matrix where each row represents a document and each column a word feature

Step 4: Compute Cosine Similarity

Cosine similarity measures the angle between vectors:

- $\cos(\theta) = (\mathbf{A} \cdot \mathbf{B}) / (||\mathbf{A}|| \cdot ||\mathbf{B}||)$
- A score close to 1 $\rightarrow$ high similarity

Step 5: Rank and Recommend

- Sort jobs based on similarity scores in descending order

Chapter 5 RESULTS

Model: Logistic Regression

Accuracy: 0.5332197614991482

	precision	recall	f1-score	support
5	0.71	0.56	0.62	18
6	0.28	0.40	0.33	20
7	0.00	0.00	0.00	2
8	0.00	0.00	0.00	3
10	0.33	0.17	0.22	6
11	0.50	1.00	0.67	1
12	0.29	0.29	0.29	7
16	0.00	0.00	0.00	5
19	0.67	0.50	0.57	4
20	0.00	0.00	0.00	1
21	0.00	0.00	0.00	3
22	0.19	0.27	0.22	15
23	0.33	0.44	0.38	9
27	0.00	0.00	0.00	1
30	0.33	0.44	0.38	9
32	0.33	0.33	0.33	9
34	0.00	0.00	0.00	3
35	1.00	1.00	1.00	1
38	0.56	0.45	0.50	22
43	0.00	0.00	0.00	4
44	0.00	0.00	0.00	1
45	0.00	0.00	0.00	1
46	0.00	0.00	0.00	1
47	0.50	0.50	0.50	2
49	0.33	0.33	0.33	3
50	0.53	0.73	0.62	11
52	0.42	0.56	0.48	9
55	0.33	0.50	0.40	2
56	0.00	0.00	0.00	4
58	0.00	0.00	0.00	2
59	0.23	0.29	0.26	17
62	0.38	0.25	0.30	12
64	0.00	0.00	0.00	1
65	0.79	0.82	0.80	191

66	0.00	0.00	0.00	8
68	0.40	0.50	0.44	4
69	0.00	0.00	0.00	3
70	0.00	0.00	0.00	2
71	0.00	0.00	0.00	4
72	0.67	0.66	0.67	50
73	0.25	0.20	0.22	5
75	0.00	0.00	0.00	3
76	0.17	0.19	0.18	21
77	0.50	0.20	0.29	5
79	0.20	0.08	0.12	12
80	0.36	0.57	0.44	7
81	0.33	0.33	0.33	3
84	0.00	0.00	0.00	1
85	0.00	0.00	0.00	1
87	0.50	0.50	0.50	6
88	0.67	0.63	0.65	52

accuracy		0.53		587
macro avg	0.26	0.27	0.26	587
weighted avg	0.53	0.53	0.53	587

Model: Random Forest

Accuracy: 0.7563884156729132

	precision	recall	f1-score	support
5	0.76	0.89	0.82	18
6	0.75	0.45	0.56	20
7	0.00	0.00	0.00	2
8	0.00	0.00	0.00	3
10	1.00	0.83	0.91	6
11	1.00	1.00	1.00	1
12	0.33	0.14	0.20	7
16	0.00	0.00	0.00	5
19	1.00	0.50	0.67	4
20	0.00	0.00	0.00	1
21	0.00	0.00	0.00	3
22	0.50	0.07	0.12	15
23	0.89	0.89	0.89	9
27	0.00	0.00	0.00	1
30	0.75	1.00	0.86	9
32	0.71	0.56	0.62	9
34	0.50	0.67	0.57	3
35	0.00	0.00	0.00	1
38	0.81	0.77	0.79	22
43	0.00	0.00	0.00	4
44	1.00	1.00	1.00	1
45	0.00	0.00	0.00	1
46	0.00	0.00	0.00	1
47	0.67	1.00	0.80	2
49	0.33	0.33	0.33	3

50	0.59	0.91	0.71	11
52	0.62	0.89	0.73	9
55	1.00	1.00	1.00	2
56	1.00	0.50	0.67	4
58	1.00	0.50	0.67	2
59	0.67	0.35	0.46	17
62	0.91	0.83	0.87	12
64	0.00	0.00	0.00	1
65	0.84	0.99	0.91	191
66	0.50	0.12	0.20	8
68	1.00	0.50	0.67	4
69	0.00	0.00	0.00	3
70	0.00	0.00	0.00	2
71	0.50	0.25	0.33	4
72	0.61	0.98	0.75	50
73	0.67	0.80	0.73	5
75	0.00	0.00	0.00	3
76	0.64	0.43	0.51	21
77	0.71	1.00	0.83	5
79	0.78	0.58	0.67	12
80	0.30	0.43	0.35	7
81	1.00	0.33	0.50	3
84	0.00	0.00	0.00	1
85	0.00	0.00	0.00	1
87	0.67	0.67	0.67	6
88	0.80	0.94	0.87	52

accuracy		0.76		587
macro avg	0.51	0.45	0.46	587
weighted avg	0.71	0.76	0.71	587

Model: SVM

Accuracy: 0.555366269165247

	precision	recall	f1-score	support
5	0.61	0.61	0.61	18
6	0.00	0.00	0.00	20
7	0.00	0.00	0.00	2
8	0.00	0.00	0.00	3
10	0.50	0.17	0.25	6
11	0.00	0.00	0.00	1
12	0.00	0.00	0.00	7
16	0.00	0.00	0.00	5
19	0.60	0.75	0.67	4
20	0.00	0.00	0.00	1
21	0.00	0.00	0.00	3
22	0.00	0.00	0.00	15
23	0.60	0.33	0.43	9
27	0.00	0.00	0.00	1
30	0.50	0.22	0.31	9
32	0.75	0.33	0.46	9

34	0.00	0.00	0.00	3
35	0.00	0.00	0.00	1
38	0.79	0.50	0.61	22
43	0.00	0.00	0.00	4
44	0.00	0.00	0.00	1
45	0.00	0.00	0.00	1
46	0.00	0.00	0.00	1
47	0.00	0.00	0.00	2
49	0.00	0.00	0.00	3
50	0.14	0.09	0.11	11
52	0.75	0.33	0.46	9
55	0.00	0.00	0.00	2
56	0.00	0.00	0.00	4
58	0.00	0.00	0.00	2
59	0.50	0.24	0.32	17
62	0.71	0.42	0.53	12
64	0.00	0.00	0.00	1
65	0.56	0.97	0.71	191
66	0.00	0.00	0.00	8
68	1.00	0.50	0.67	4
69	0.00	0.00	0.00	3
70	0.00	0.00	0.00	2
71	0.00	0.00	0.00	4
72	0.48	0.90	0.62	50
73	1.00	0.20	0.33	5
75	0.00	0.00	0.00	3
76	0.25	0.10	0.14	21
77	0.00	0.00	0.00	5
79	0.00	0.00	0.00	12
80	0.60	0.43	0.50	7
81	0.00	0.00	0.00	3
84	0.00	0.00	0.00	1
85	0.00	0.00	0.00	1
87	1.00	0.50	0.67	6
88	0.67	0.73	0.70	52

accuracy			0.56	587
macro avg	0.24	0.16	0.18	587
weighted avg	0.45	0.56	0.47	587

Cross-validation scores (StratifiedKFold):
Logistic Regression: Mean=0.5566, Std=0.0072
Random Forest: Mean=0.7333, Std=0.0048
SVM: Mean=0.5600, Std=0.0055

Chapter 6

CONCLUSION

6.1 SUMMARY:

The Job Recommendation System is a machine learning-based application designed to match job seekers with relevant job opportunities based on their resumes, skills, and preferences. It uses techniques like **content-based filtering** to analyze the similarity between user profiles and job descriptions by converting them into vector form using **TF-IDF** and calculating **cosine similarity** scores. Additionally, classification algorithms such as **Logistic Regression** or **Random Forest** are used to predict suitable job categories for users based on features like qualifications, experience, and technical skills. The system helps automate and personalize the job search process, providing accurate and efficient recommendations that save time for both candidates and recruiters. Evaluation metrics such as accuracy, precision, recall, and F1-score ensure the effectiveness of the models used. Overall, the system enhances the job-matching process by delivering intelligent and data-driven suggestions.

6.2 Future Enhancements :

While the current job recommendation system provides effective and personalized job suggestions using machine learning algorithms, there is significant scope for future enhancements to improve its accuracy, scalability, and user experience. One major improvement could be the integration of **deep learning models** such as **BERT** or **Doc2Vec** to better capture the contextual meaning of resumes and job descriptions, allowing for more semantically accurate recommendations. Additionally, incorporating **collaborative filtering** techniques could enhance recommendations by considering user behavior patterns and preferences, such as job clicks, applications, and ratings. Implementing **real-time recommendation updates** based on user interaction history can also make the system more dynamic and adaptive. Another enhancement could involve developing a **web or mobile interface** with user authentication, resume parsing, and interactive dashboards for job tracking. Furthermore, using **location-based filtering** and **salary prediction** models can provide more targeted and useful results. Finally, the system can be scaled for multilingual support and extended to include career guidance features like skill-gap analysis and personalized upskilling.

This would make the model more accessible and practical for end-users.

CHAPTER 7

REFERENCES

1. Lops, P., De Gemmis, M., & Semeraro, G. (2011).
Content-based Recommender Systems: State of the Art and Trends.
In: Ricci, F., Rokach, L., Shapira, B., Kantor, P.B. (eds) *Recommender Systems Handbook*. Springer.
https://doi.org/10.1007/978-0-387-85820-3_3
2. Aggarwal, C. C. (2016).
Recommender Systems: The Textbook.
Springer.
ISBN: 978-3-319-29658-0
3. Jiawei Han, Micheline Kamber, and Jian Pei (2011).
Data Mining: Concepts and Techniques (3rd ed.).
Morgan Kaufmann Publishers.
4. Bennett, J., & Lanning, S. (2007).
The Netflix Prize.
In *Proceedings of KDD Cup and Workshop, 2007*.
<https://www.cs.uic.edu/~liub/KDD-cup-2007.html>
5. Gopalan, P., & Hofman, J. M. (2015).
Scalable Recommendation with Hierarchical Poisson Factorization.
In *Proceedings of the Thirty-First Conference on Uncertainty in Artificial Intelligence (UAI-15)*.
<https://arxiv.org/abs/1311.1704>
6. Indeed.com & LinkedIn datasets:
Used for extracting real-world job listings and resumes to train and test ML models. (If used in your project.)
7. Scikit-learn Documentation.
Machine Learning in Python.
<https://scikit-learn.org/>
8. Kaggle Datasets – Job Recommendation Dataset.
<https://www.kaggle.com/>
(Search for job recommendation-related datasets, resume classification datasets, etc.)

CODE

RESUME BASED JOB RECOMMENDATION (MULTIPLE INPUT)

import pandas as pd

from sklearn.feature_extraction.text import TfidfVectorizer

from sklearn.metrics.pairwise import cosine_similarity

JOB DATASET

jobs = {

"job_title": [

"Data Scientist", "Web Developer", "ML Engineer",

"Data Analyst", "Software Engineer", "AI Engineer"

],

"job_description": [

"Python machine learning data analysis statistics",

"HTML CSS JavaScript web development frontend backend",

"Deep learning neural networks artificial intelligence python",

"SQL Excel data visualization business analysis",

"Java C++ software development coding programming",

"Artificial intelligence machine learning deep learning python"

]

}

jobs_df = pd.DataFrame(jobs)

LOOP FOR MULTIPLE INPUTS

while True:

print("\nEnter your resume (or type 'exit' to stop):\n")

resume_text = input()

if resume_text.lower() == 'exit':

print("Program Ended ✔")

break

TF-IDF

all_text = jobs_df['job_description'].tolist()

all_text.append(resume_text)

tfidf = TfidfVectorizer(stop_words='english')

tfidf_matrix = tfidf.fit_transform(all_text)

Similarity resume_vector = tfidf_matrix[-1]

```

job_vectors = tfidf_matrix[:-1]
similarity = cosine_similarity(resume_vector, job_vectors)
# Top jobs
top_indices = similarity[0].argsort()[::-1][:3]
print("\n TOP JOB RECOMMENDATIONS:\n")
for i in top_indices:
    print("Job Title:", jobs_df.iloc[i]['job_title'])
    print("Match Score:", round(similarity[0][i], 2))
    print("-"*40)

```

OUTPUT:

Python machine learning AI data analysis

✓ Output → Data Scientist, ML Engineer